

COMPUTER SCIENCE TRIPOS Part IB – 2022 – Paper 4

2 Compiler Construction (tgg22)

This question involves the derivation of “stack machines” using the CPS transformation.

Tail recursion

(a) Consider the following OCaml code of type

```
add_right : int list -> int
```

that returns the sum of the integers in its argument list.

```
let rec add_right l =  
  match l with  
  | [] -> 0  
  | h::tl -> h + (add_right tl);;
```

Explain why this code, as presented, is not tail recursive.

[2 marks]

Answer: The last operation applied in a recursive flow of control is not the recursive call itself (it is an addition).

Continuation
passing style
(CPS)

(b) Use the CPS transformation to rewrite `add_right` to a function that could be given the type

```
add_right_cps : int list -> (int -> int) -> int
```

[6 marks]

Answer: Here is the simplest way of expressing this function.

```
let rec add_right_cps l c =  
  match l with  
  | [] -> c 0  
  | h::tl -> add_right_cps tl (fun v -> c(h + v));;
```

Note that the most general type is actually

```
add_right_cps : int list -> (int -> 'a) -> 'a
```

but that the desired type is an instance of this type. However, this most general type hints at a more general approach to the CPS transformation!

Represent
continuations as
data

(c) Apply defunctionalisation to your code for `add_right_cps`. That is, define a (non-functional) data type `cnt` and a transformed function `add_right_dfc` of type

```
add_right_dfc : int list -> cnt -> int
```

[6 marks]

Answer: One approach would be as follows.

```
type cnt = ID | CNT of int * cnt ;;

let rec add_right_dfc l c =
  match l with
  | [] -> apply c 0
  | h::tl -> add_right_dfc tl (CNT(h, c))
and apply c v =
  match c with
  | ID -> v
  | CNT (h, c) -> apply c (h + v);;
```

But note, we don't need mutual recursion in this case since `apply` does not call `add_right_dfc`. So, we could write this as

```
let apply c v =
  match c with
  | ID -> v
  | CNT (h, c) -> apply c (h + v);;

let rec add_right_dfc l c =
  match l with
  | [] -> apply c 0
  | h::tl -> add_right_dfc tl (CNT(h, c));;
```

Alternatively, we can see that the above definition of `cnt` looks very much like the definition of integer lists. So, we could write instead

```
type cnt = int list ;;

let apply' c v =
  match c with
  | [] -> v
  | h::c -> apply' c (h + v);;

let rec add_right_dfc' l c =
  match l with
  | [] -> apply' c 0
  | h::tl -> add_right_dfc' tl (h::c);;
```

The CPS
transform

- (d) The function `add_right` from Part (a) could be generalised to the following function.

```
let rec fold_right f l accu =
  match l with
  | [] -> accu
  | a::l -> f a (fold_right f l accu);;
```

For simplicity, we will treat this code as if it had the type

```
fold_right : (int -> int -> int) -> int list -> int -> int
```

and not worry about polymorphism. Rewrite this program using the CPS transformation. Justify your treatment of the variable `f`. What problems might you encounter in attempting to defunctionalise your CPS version? [6 marks]

Answer:

When we apply the CPS transformation

```
let rec fold_right_cps f l accu c =  
  match l with  
  | [] -> c accu  
  | a::l -> fold_right_cps f l accu (fun v -> f a v c);;
```

we must assume that all of the source code has been transformed using CPS. In other words, we must assume that any function passed to this code via `f` has also been transformed using CPS.

If we attempt to defunctionalise to something like this

```
type cnt = ID | CNT of int * cnt ;;  
  
let apply c v =  
  match c with  
  | ID -> v  
  | CNT (a, c) -> ?APPLY? (CNT(v, c))(f a) ;.  
  
let rec fold_right_dfc f l accu c =  
  match l with  
  | [] -> apply c 0  
  | a::l -> fold_right_dfc f l accu (CNT(a, c))
```

where we have to use a application function `?APPLY?` that must somehow understand the representations of continuations associated with the argument `f`. Since many different functions can be passed via `f` we need some kind of “global” definition of continuations and their application.

This is simply pointing out the limitations of the CPS transform as it has been presented in lectures. The curious student may want to think about a more general approach.
